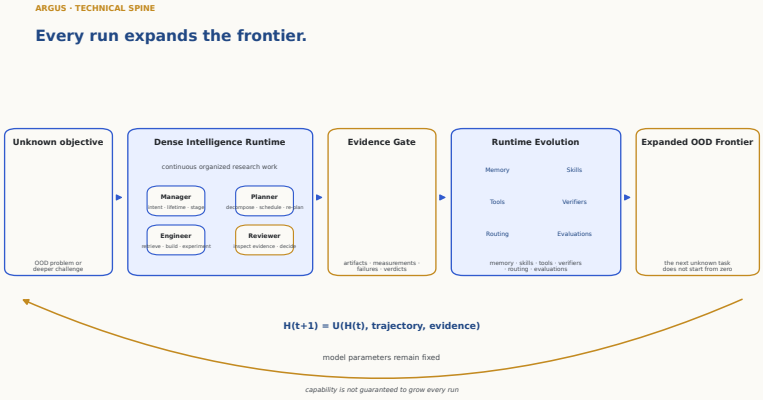
ARGUS

Autonomous Research Generation and Understanding System

Dense Intelligence for an Expanding Research Frontier



Argus is a research infrastructure that operates a capable coding-and-reasoning model as an autonomous research system over long horizons — hours to days — against real, public benchmarks and open research problems. This report documents the system as engineered: a three-plane architecture, the research runtime and mission lifecycle, the state and knowledge systems, the agent decision interfaces, the experiment and resource infrastructure, the reliability and recovery model, and the evidence architecture that keeps every measurement auditable. It reports the current public results across six arenas and a 41-paper research portfolio, each a scoped claim under its stated protocol and evidence tier. It is an engineering report, not a results paper: it does not claim universal state of the art, an accepted autonomous publication, or correctness outside the stated protocols.

Contents

ACT I · THE DENSE-INTELLIGENCE PROBLEM

Research intelligence is sparse in time.

Long-horizon research requires continuity, execution, verification, and durable state—not merely a longer context window.

1 Executive Thesis

Argus sustains dense research intelligence, turns each run's evidence into runtime capability, and carries that capability into the next unknown task. Every run expands the frontier.

Model intelligence is sparse in time. A capable coding-and-reasoning model is brilliant for the length of a single call and then stops: the reasoning that produced an insight evaporates when the context is dropped or lossily compacted, and the next call begins again with no durable trace of what was learned. Long-horizon research is the opposite of a single call. It is thousands of coupled decisions sustained over hours to days, where the value is not in any one clever step but in keeping judgment, execution, and verification connected across all of them. A longer context window does not close this gap; it only postpones the moment the episode ends.

Argus makes intelligence *dense* by running four persistent, model-driven roles as a continuous loop over persisted project state. A Manager fixes intent and lifetime, a Planner decomposes and schedules, an Engineer retrieves and builds and experiments, and a Reviewer inspects the evidence and decides. Because the loop never dissolves back into a single stateless call, decision, execution, and verification stay coupled: the system can carry a thread of research reasoning far past the horizon at which an episodic agent would have forgotten why it started.

Continuity alone is not enough — it must compound. Every run deposits verified evidence, and that evidence updates the system's *runtime state*: its memory, skills, tools, verifiers, routing, and evaluations. This update is gated by the Reviewer, so only checked results change the runtime, and it happens with the model's parameters held fixed — $H(t{+}1) = U(H(t), \text{trajectory}, \text{evidence})$. The system grows more capable not by retraining a model but by accumulating audited, reusable capability around it.

The consequence is out-of-distribution reach. When the next task arrives, the runtime it inherits already carries the skills, tools, and verifiers earned on prior problems, so the next unknown objective does not start from zero. Each expansion of verified capability enlarges the frontier of problems the system can attempt — the expanding-frontier chain of Figure 1.

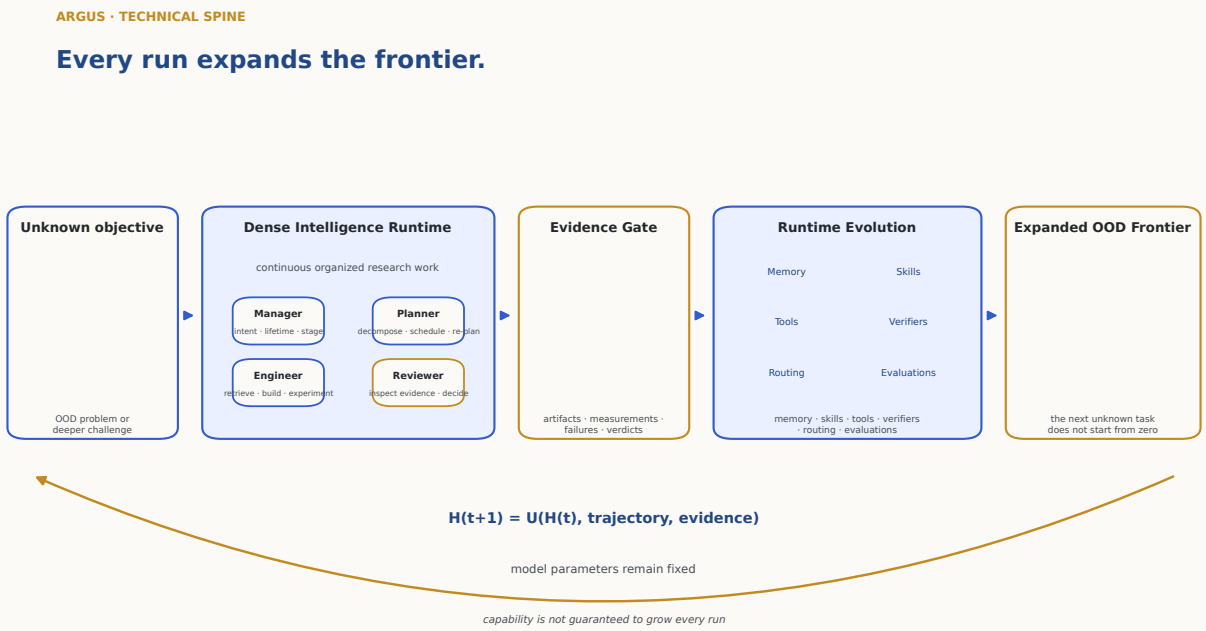


Figure 1. The expanding-frontier spine. An unknown, out-of-distribution objective enters a dense-intelligence runtime driven by four persistent roles; verified work passes an evidence gate; the gate updates durable runtime state; and the enlarged runtime meets the next unknown task from a higher floor. Model parameters remain fixed, and capability is not guaranteed to grow on every run.

What this report claims	What it does not claim
Argus is engineered as a persistent, role-separated runtime that keeps decision, execution, and verification coupled over long horizons.	That the infrastructure is more capable than the model, or substitutes heuristics for the agent's scientific judgment.
Each run's reviewer-verified evidence updates durable runtime state — memory, skills, tools, verifiers, routing, evaluations — with model parameters fixed.	Monotone empirical capability growth: a run may fail, return only a negative result, or add nothing verified.
Six public arena results, each a scoped claim under its stated protocol, hardware, and evidence tier.	Universal state of the art, or superiority over all human researchers or other systems.
A 41-paper portfolio compared only to human-authored literature, reported as a de-duplicated inventory.	An accepted autonomous publication, or a count of accepted papers.
Reliability and auditability by construction: bounded budgets, one completion authority, an append-only evidence tape.	Correctness, safety, or performance outside the exact protocols each result is measured under.

Table 1. The claim boundary the rest of the report is held to.

Argus publishes a live public record [1]. This report reproduces its six current arena results (Section 11, Figure 7), each with the protocol and hardware it was measured under and its published reference: NVIDIA SOL-ExecBench (B200, 101 kernels; global rank #6 with two head-to-head wins over Recursive [7]), nanochat on B200 (0.9636 BPB vs. human SOTA 0.9646) and H100 (0.9855 vs. 0.9879 BPB), the nanoGPT speedrun (79.77 s vs. the same-device human record 80.18 s, N=10), AARRI-Bench (63/82, 76.8% vs. the paper-reported best 68.3%), and Arbor's site-reported system comparison; two of the six — the nanoGPT speedrun and nanochat on B200 — additionally

carry committed artifact-digest records, while the other four remain website snapshots, and that distinction is kept visible throughout (Section 10). Separately, the public research page [2] lists a portfolio of **41 papers** — 35 manuscripts and 6 drafts across six research programs (Section 12, Figure 8) — compared only to human-authored literature and baselines, and reported as a de-duplicated inventory rather than a count of accepted papers.

Design objective, not a measured guarantee

"Every run leaves the system more capable" is the design objective: a run may fail, produce only a negative result, or add no verified capability. The report does not claim monotone empirical capability growth.

2 Dense Intelligence

Design objective

Keep research intelligence *dense* in time: hold decision, execution, and verification coupled and continuous over persisted state, so that a long-horizon program behaves as one sustained line of reasoning rather than a sequence of disconnected bursts.

2.1 Definition

By *dense intelligence* we mean sustained, coupled research work: at every point in a long run the system is making decisions, carrying them into real execution, and verifying the outcome against evidence — and the thread that connects these is never broken by the end of a model call. Sparse intelligence is the default failure mode of a capable model left to run long: it is brilliant in bursts and empty in between, because each burst ends and its working state is discarded. Density is not about being busy; it is about the fraction of elapsed research time in which genuine decisions are being made, executed, and checked, rather than lost to context recovery, idle polling, or unverifiable self-report. Figure 2 contrasts episodic research, where useful work is separated by context-recovery gaps, with the continuous role loop that runs over persisted project state.

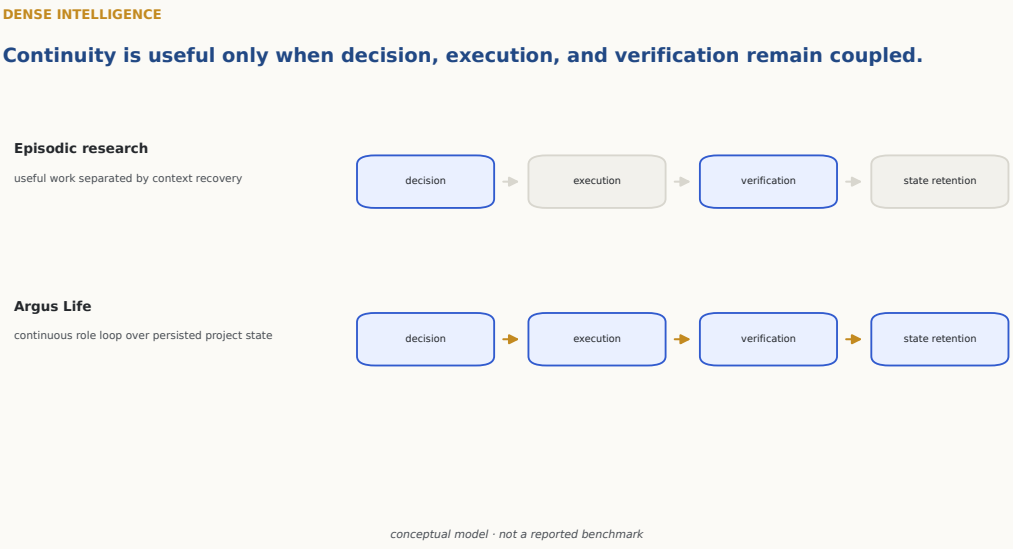


Figure 2. Dense intelligence as continuity of coupled work. Episodic research recovers decision and verification repeatedly across context gaps; Argus keeps decision, execution, verification, and state retention coupled over a persisted project. The schematic is a conceptual model, not a reported benchmark.

2.2 Continuity is not token volume

The tempting shortcut is to equate continuity with a larger context window: keep more tokens live and the run will stay coherent. It will not. A longer window lengthens a single episode but does nothing to couple decision to execution to verification, and it still terminates — at which point the accumulated working state is compacted or dropped and the next episode restarts cold. Token volume measures how much text a model can attend to at once; density measures how much of a research horizon is spent on verified, decision-bearing work. A run can consume enormous context and remain sparse — re-reading the same files, re-deriving the same dead ends — because none of that volume is gated by an independent check or retained as reusable capability. Continuity that matters is structural: it comes from persisting state outside the model and cycling roles over it, not from holding more tokens inside the model.

2.3 A conceptual density model

To make the idea precise enough to reason about — and no more — we summarize it as a time-averaged density over a research horizon T:

$$\rho_{\mathrm{DI}}(T) = \frac{1}{T}\int_0^T \lambda(t)\,\eta_d(t)\,\eta_x(t)\,\eta_v(t)\,dt. \tag{1}$$

The integrand multiplies a rate of research activity by three efficiencies, so that time spent busy but undirected, or active but unverified, contributes little.

Symbol	Meaning
T	The research horizon over which density is averaged — hours to days, not a single call.
$\lambda(t)$	Instantaneous rate of research activity: how much work the system is attempting at time t.
$\eta_d(t)$	Decision efficiency: the fraction of that activity that is a genuine decision advancing the frontier rather than repetition.
$\eta_x(t)$	Execution efficiency: the fraction of decisions carried into real execution on real tools, data, and hardware.
$\eta_v(t)$	Verification efficiency: the fraction of executed work that is independently checked against evidence.

Table 2. The five symbols of the conceptual density model.

Read this way, the episodic failure mode is exactly a collapse of one or more efficiencies to near zero for long stretches: high λ but low η_d (busy, not deciding), or non-trivial η_d and η_x but low η_v (acting, not verifying). The design of Argus is an attempt to hold all three efficiencies away from zero across the whole horizon by construction — persistent roles for decision and verification, real execution for η_x, and an evidence gate for η_v.

2.4 What is and is not measured

The model above is a lens, not an instrument. ρ_{DI} is an explanatory construct, not a reported benchmark metric. It does not, in itself, establish that Argus is universally superior to human researchers or other systems. No number in this report is a measurement of ρ_{DI}: the integrand's efficiencies are not instrumented as a live scalar, and we make no attempt to publish one. What *is* measured is reported elsewhere under explicit protocols — the six public arena results and the 41-paper portfolio of Section 11 and Section 12, each a scoped claim under its stated evidence tier. Dense intelligence is the design goal those results are evidence *for*; it is not itself a score, and it is not a claim of general superiority.

3 Why Episodic Agents Stall

Design objective

Diagnose why a capable model, run long, degrades into busy sparsity — and derive the design requirements that a system must meet to make research *compound* across runs instead of restarting from zero.

3.1 Model capability is not system capability

It is easy to assume that a more capable model is, by itself, a more capable research system. A stronger model does raise the ceiling of any single step: it reads code more accurately, writes better prose, and reasons further in one call. But a research *system* is judged over thousands of steps sustained across hours to days, and that horizon is governed by properties the weights do not supply: whether the thread of work survives the end of a call, whether claims are checked before they are believed, whether what was learned is retained and reused. Model capability is per-call; system capability is cross-call. Treating them as the same thing is why "give a big model a hard problem and wait" fails — the missing capability lives in the substrate around the model, not in the model.

3.2 Episodic work pays context-recovery cost

Left to run long, a model works in episodes. Each episode accumulates context until the window overflows and the transcript is compacted or dropped; the next episode then reconstructs where it was — re-reading the same files, re-deriving decisions it already made, re-discovering dead ends it already ruled out. This context-recovery cost is pure overhead: activity with near-zero decision efficiency. It is not a prompt-quality problem that a better instruction removes; it is a structural property of unbounded session growth. The longer the horizon, the larger the fraction of elapsed time the system spends paying this tax instead of advancing the frontier.

3.3 Unbounded context is not institutional memory

The obvious response — never drop context, just make the window bigger — solves the wrong problem. An ever-growing transcript is not memory. Institutional memory is curated, addressable, and reusable: a specific technique that worked, a baseline that was rejected and why, a verifier that caught a fake result. A monolithic transcript is none of these; it is an undifferentiated log in which the few reusable facts are buried among the reasoning that produced them, and it is precisely what lossy compaction shreds first when the window finally overflows. Durable capability has to live *outside* the model, in persisted state that can be written deliberately, retrieved selectively, and carried into a future run.

3.4 Process data disappears when only the final artifact survives

Most of the value of a research run is in its process, not its final artifact. The manuscript or merged patch records the destination; it discards the map of how the work got there — what was attempted and failed, which measurement was found contaminated, why one design was chosen over another. When a system keeps only the final artifact, that process knowledge is thrown away at the end of every run, so the next run cannot stand on it and is free to repeat the same mistakes. A system that is meant to compound must persist the trajectory and the verdicts, not just the deliverable, and must treat those process records as first-class, auditable state.

3.5 Design requirements for compounding research

These failure modes are not independent bugs; together they define what a long-horizon research system must provide. Five requirements follow directly, and each is met by a concrete mechanism in the architecture that Act II documents (Section 4).

R1. Persistent objective. A durable, operator-anchored objective that survives restarts and upgrades, so the system always knows what it is pursuing and never silently re-plans a campaign

from a stale premise.

R2. Role separation. Distinct model-driven roles for decision, execution, and verification, so no single stateless call owns all three and completion is never a self-report from the role that did the work.

R3. Bounded context. Per-session context held to an explicit bound and re-seeded from curated working memory, so the runaway compaction that drives context-recovery cost cannot occur.

R4. Independent evidence. Progress and completion judged by an authority that inspects artifacts and execution logs against evidence, rather than inferred from activity, filenames, or token volume.

R5. Reusable runtime state. Verified evidence deposited as durable, addressable capability — memory, skills, tools, verifiers, routing, evaluations — that the next unknown task inherits instead of starting from zero.

Act II turns these five requirements into a running system: a three-plane architecture that separates decision authority, execution, and evidence, and the runtime that keeps them coupled over a persisted project.

4 System Architecture

Design objective

Give every component one home in a three-plane model so that decision authority, work execution, and evidence recording are physically separated and independently auditable. Judgment is confined to the control and execution planes; the evidence plane records the run but never decides it.

Argus is organized as three cooperating planes. The **control plane** holds the roles and scheduler that decide *what to run and whether it is done*. The **execution plane** runs *one mission at a time* against real tools, providers, and budgets. The **evidence plane** records the run on an append-only tape with usage accounting, credential redaction, and provenance digests. Figure 3 shows the planes and their interfaces; Figure 4 is the accepted system schematic. The end-to-end control path is a single spine: `CLI` → `Manager.divide` (vertical selection) → runtime wiring → `LifeSupervisor` (mission scheduler) → `SkillLoop` → `SupervisedEngineer` ↔ Reviewer-until-done, resting on the core services, the single anti-stall mechanism, the pluggable verticals, and the provider backend.[1]

4.1 Control plane: decision authority

The control plane is where scientific and scheduling authority live. The **Manager** (`manager/_core.py`, `manager/front_door.py`, `manager/dispatch.py`) is the single operator entry point: one grounded model call routes free text to chat or task, selects the task shape (*vertical*) or authors a new data domain, and commits that choice durably. It is also the *sole* authority for pipeline-stage transitions — it owns the `current_stage` field, and the other roles may only advise a change. The **Planner** (`planner/planner.py`) is the forward scheduler: it proposes the next batch of work as a task DAG when the backlog empties. The **LifeSupervisor** (`life/supervisor/_core.py`) is the 7×24 outer loop that claims backlog items, runs missions, accounts cost, and plans next. These components decide; they do not themselves touch a provider.

4.2 Execution plane: one mission at a time

The execution plane runs a single mission. `SkillLoop` (`loop.py`) is the per-mission glue: it matches or distills a reusable skill, builds the Engineer prompt, and drives the Engineer↔Reviewer round loop. The **Engineer** (`engineer/runner.py`) performs one round of real work — literature search, code, experiments, or writing — and the **Reviewer** (`reviewer/_core.py`) returns a structured verdict. Every provider call, regardless of role, is issued through a single application-level gateway

[1] Spine and module map: `docs/ARCHITECTURE.md`. Plane assignment cross-referenced in Appendix D.

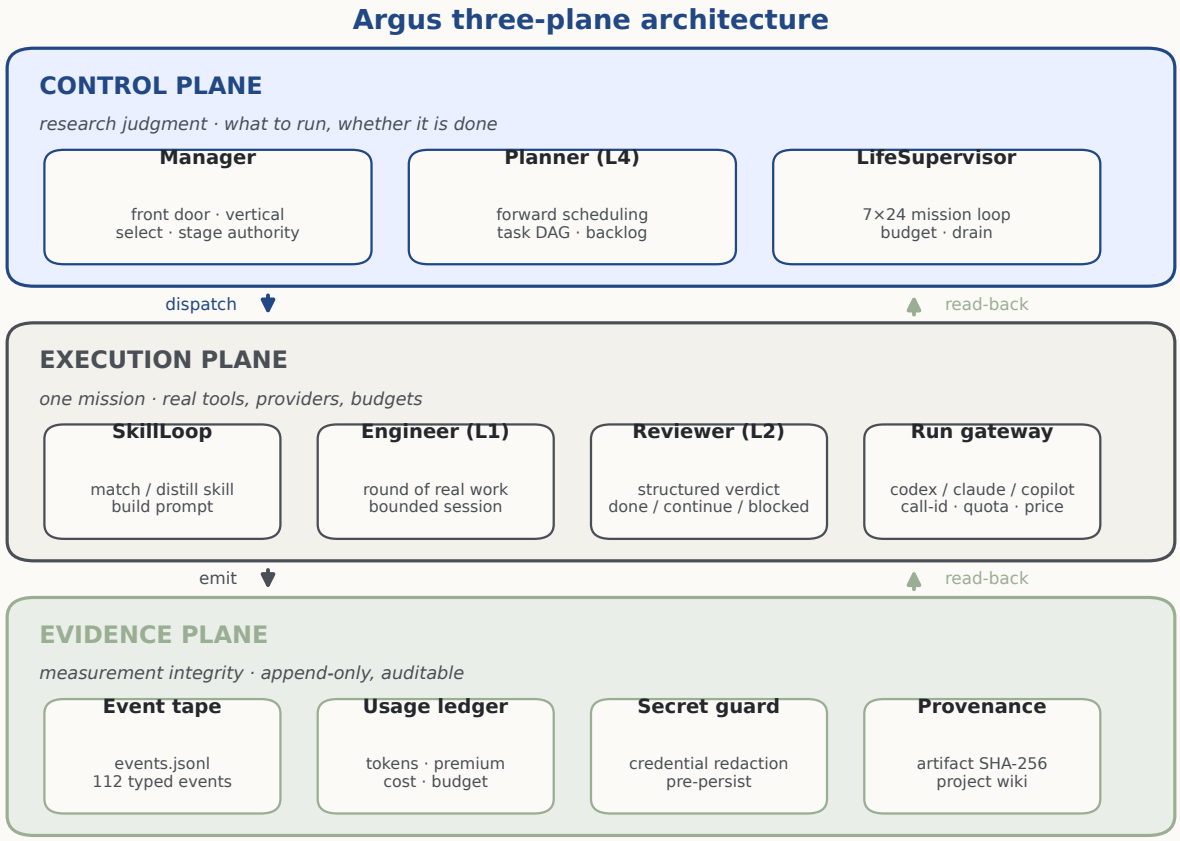


Figure 3. The three-plane architecture. Control-plane roles dispatch work into the execution plane, which emits typed events into the evidence plane; the evidence plane is read back by both other planes but holds no decision authority. Drawn deterministically from the module map (Appendix D).

(`core/run_gateway.py`), which assigns a `call_id`, propagates the resume-thread identity, and stamps timing — one interception point for tracing, metrics, and policy. The concrete backends (Codex, Claude Code, GitHub Copilot CLI) sit behind an adapter (`adapters/agent_cli_backend.py`); a deterministic in-memory backend (`adapters/memory_backend.py`) drives tests.

4.3 Evidence plane: measurement integrity

The evidence plane records the run and enforces the run's integrity invariants (Section 10). Its spine is a per-project append-only event tape, `events.jsonl` (`core/paths.py`), carrying the 112 typed event types. Alongside it sit the usage ledger (`core/usage.py`), the credential-redaction pass applied before persistence (`core/secret_guard.py`), the provenance and artifact-corroboration records (`skills/provenance.py`, `webapi/artifacts.py`), and the project wiki of immutable, evidence-cited pages (`argus_skill/wiki/`). Nothing in this plane makes a scientific decision; it exists so that a decision made elsewhere can be checked.

4.4 Module map

Table 3 maps each area of the live tree to its plane and files. The map is authoritative in `docs/ARCHITECTURE.md` and is kept in sync with the code.

4.5 What is deliberately not in the spine

Two things are kept off the default path. The research-paper pipeline (the `research` vertical and its paper machinery) is lazy-loaded only when a project's vertical is `research`; it is not part of the metric-reproduction product. And a **Curator** role (skill-pool maintenance, team-leaderboard distillation) runs only in team mode. Both are described where they are used (Sections 6, 12) rather

Area	Plane	Primary file(s)
Entry / CLI	control	`apps/cli/_parser.py`, `apps/cli/_core.py`, `__main__.py`
Manager	control	`manager/_core.py`, `manager/front_door.py`, `manager/dispatch.py`
Planner	control	`planner/planner.py`, `planner/planner_schema.json`
Mission scheduler	control	`life/supervisor/_core.py`
Daemon	control	`daemon/life_worker.py`, `daemon/handoff.py`
Skill loop	execution	`loop.py`
Engineer	execution	`engineer/runner.py`, `engineer/checkpoint.py`
Reviewer	execution	`reviewer/_core.py`, `reviewer/reviewer_schema.json`
Provider gateway	execution	`core/run_gateway.py`, `adapters/agent_cli_backend.py`
Anti-stall (meta)	execution	`regime_jump/` (saturation, flow, ledger, meta-prompter)
Verticals	execution	`verticals/` — nanochat, nanogpt_speedrun, kernelbench, research, ...
Core services	evidence	`core/paths.py`, `core/usage.py`, `core/pricing.py`, `core/models.py`
Integrity	evidence	`core/secret_guard.py`, `skills/provenance.py`
Event catalog	evidence	`core/event_catalog.py`, `core/event_payload_schemas.json`
Cockpit	(all)	`webapi/server.py`, `frontend/tui/`, `frontend/web/`

Table 3. Module map, by plane. Areas marked "(all)" observe across planes but hold no decision authority. Full table: `docs/ARCHITECTURE.md`.

than as spine components.

5 Research Runtime and Mission Lifecycle

Design objective

Run one mission at a time, back to back, for days — surviving restarts, upgrades, and provider outages — without ever re-deciding a committed campaign or killing a mission mid-flight. The unit of scheduling is a mission; the unit of durability is the mission boundary.

The research runtime is the 7×24 loop that turns a durable objective into a stream of missions. Figure 5 shows the lifecycle and its recovery edges. This section describes the states, the process model, and the restart/resume semantics that make a long campaign durable.

5.1 From objective to missions

An objective enters through the Manager, which *divides* it: it selects one vertical and commits it durably. From that point the scheduler *trusts* the persisted vertical and never re-classifies the task — a single division decision, not a per-round guess.[2] A task's *lifetime* is likewise an explicit, Manager-decided property: most team tasks default to `STANDING` (run 7×24) and are persisted atomically to `continuous.json`; a task with a natural one-shot endpoint is `BOUNDED`. An operator may force `BOUNDED` at daemon start with `-bounded`.

The Manager also owns pipeline-stage progression. A vertical supplies an ordered stage template (`STAGE_ORDER`); the Manager advances `current_stage` through it via a stage decision whose `action` is one of `advance`, `hold`, `rollback`, or `complete`, with a `source` field that records *why* (a model decision, or one of several fail-safe holds). The Planner, Engineer, and Reviewer may advise a stage change but never write it (Section 7).

[2] `manager/_core.py`; the scheduler reads the committed vertical rather than re-running the Manager.

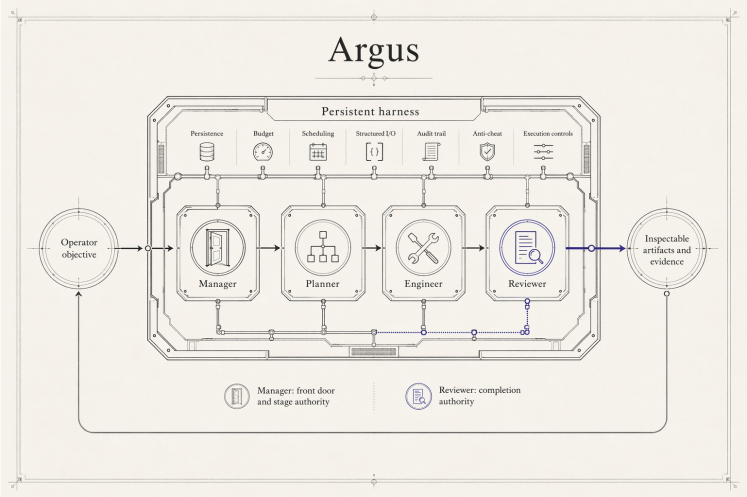


Figure 4. System schematic (not a measured performance plot): an operator objective enters the persistent runtime; the Manager, Planner, Engineer, and Reviewer sit inside it as the four roles that carry every research judgment; and inspectable artifacts and evidence exit back to the operator.

5.2 The mission scheduler

The `LifeSupervisor` runs a synchronous outer loop: it claims the next backlog item, injects a non-authoritative recency memory preamble without polluting the raw objective, runs the mission through the execution plane, accounts the cost against budget gates, writes a journal entry, and — when the backlog empties in continuous mode — invokes the Planner for the next batch. The backlog claim (`pending→running`) is atomic, so two workers can never take the same item; orphaned `running` items from a prior crash are reaped on startup.[3] The scheduler is deliberately synchronous — one mission at a time — which is what makes the mission boundary a clean, well-defined point for stopping, upgrading, and resuming.

Every mission ends in a first-class terminal or paused status, never an unbounded spin. Terminal outcomes include `done`, `blocked`, `max_rounds`, `no_progress`, and `error`; paused outcomes include `paused_budget`, `paused_provider_cooldown`, `paused_provider_fence`, and `infra_blocked` (the full taxonomy is in Appendix A). A paused mission is not a failure: it records that the campaign is correctly waiting on an external condition and should be rechecked later.

5.3 Process model: the daemon

For unattended operation the runtime runs as a detached daemon worker (`daemon/life_worker.py`, roughly 1,800 lines) wrapping the supervisor. Two properties make it safe to run continuously.

Drain to the mission boundary.
On `SIGTERM` or `SIGINT` the worker stops cleanly at the next mission-tick boundary rather than interrupting a live mission, so an in-progress evaluation or a result being recorded is never severed.

Two-layer concurrency guard.
A run is protected by both the atomic backlog claim and a per-project `daemon.pid` lock (`core/daemon_lock.py`), so a second daemon cannot drive the same project. Continuous-mode configuration is coordinated per project through `continuous.json`, which the worker hot-reloads.

5.4 Upgrades: blue/green handoff and identity-checked resume

A 7×24 system must be upgradable without losing a campaign. Argus handles a source change with a **blue/green self-handoff**: the worker detects a change in its own source signature, spawns a candidate child process under an environment-driven handoff protocol with bounded generation count and a minimum inter-handoff interval, and hands the campaign over at a clean boundary

[3] `life/supervisor/_core.py`; atomic `Backlog.claim_next` and startup orphan reaping.

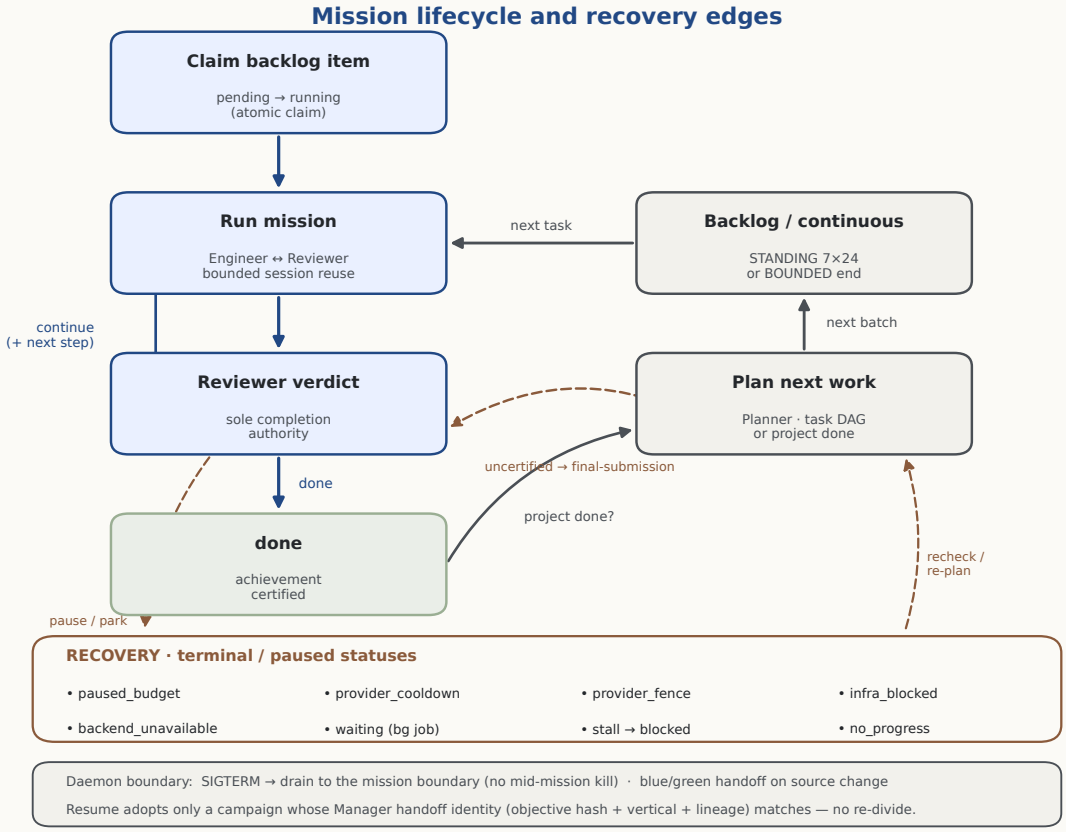


Figure 5. Mission lifecycle and recovery edges. The primary spine (indigo) claims a backlog item, runs an Engineer↔Reviewer mission, and acts on the Reviewer verdict; the plan-next loop (grey) refills the backlog; recovery edges (terracotta) route paused and blocked missions to first-class statuses and back. The daemon boundary governs draining, handoff, and identity-checked resume.

(`daemon/handoff.py`). The Manager-to-daemon handoff *identity* (objective hash, vertical, lineage generation) is persisted and recoverable from the immutable event tape under a shared file lock.

Resume is identity-checked. A `-resume-continuous` start adopts only a persisted campaign whose handoff identity matches; a mismatched or missing identity forces a real Manager division rather than blindly re-attaching. This is what prevents an upgrade or a crash recovery from silently re-planning a campaign from stale state.[4]

5.5 Crash and reboot recovery

Crash and reboot restart is owned by the service manager, not bespoke logic: the `systemd -user` unit (Section 14) restarts the worker on failure with a bounded budget and drains to the mission boundary on stop, and `loginctl enable-linger` carries it across logout and reboot. The durable answer to a box restart is a standard, inspectable unit file plus the identity-checked resume above — not a custom keep-alive of unknown behavior.

6 State, Memory, and Knowledge Systems

Design objective

Persist every mission-relevant fact on disk so that the runtime can recover at round and mission boundaries. Working memory is a small, hard-capped structured state; long-term knowledge is a set of reusable skills and an immutable, evidence-cited wiki. Provider sessions are bounded execution caches, not the sole system of record.

Long-horizon autonomy fails when state is trapped in a growing model context. The Argus answer

[4] `daemon/life_worker.py` handoff-identity read/write; `daemon/handoff.py` generation and interval bounds.

Path (per project)	Owner	Content	Mutability
`events.jsonl`	evidence plane	canonical typed-event timeline	append-only
`backlog.jsonl`	scheduler	pending / running / done work items	atomic claim
`continuous.json`	scheduler	STANDING lifetime + campaign config	rewrite
`inbox.jsonl`	operator	queued operator messages / answers	append + drain
`skills/`	skill system	project-layer distilled skills	versioned
`checkpoint.json`	Reviewer	hard-capped curated working memory	atomic rewrite
`daemon.pid`	daemon	concurrency lock	exclusive
`daemon.status.json`	daemon	liveness / status snapshot	rewrite
`identity.md`†	global	author / machine identity	operator
`skills/`†	global	shared skill library	versioned
`config.json`†	global	operator knob overrides	operator

Table 4. Principal persisted state (abridged; full map in Appendix B). The event tape is the only append-only source of truth; everything else is derivable or resettable working state. † under the global root `~/.argus-skill/`.

is to keep the durable truth on disk and to treat each provider session as bounded and disposable. This section describes the on-disk layout, the working-memory model, the session model, and the two long-term knowledge systems.

6.1 On-disk layout

Core path helpers define the global and per-project roots (`core/paths.py`); runtime composition derives the checkpoint path inside that project state directory (`apps/_runtime.py`). A global root, `~/.argus-skill/` (overridable via `ARGUS_SKILL_HOME`), holds cross-project identity, shared skills, and an operator knob store; each project lives under `projects/<fingerprint>/`, where the fingerprint is derived from the project working directory (empty, `.`, and `/` fingerprints are rejected). Table 4 lists the principal state files, their owner, and their mutability. The canonical timeline is the append-only `events.jsonl`; other files (backlog, continuous configuration, inbox, daemon status) are derived views or working state that can be rebuilt or safely reset.

6.2 Working memory and bounded provider sessions

Argus does *not* depend on one unbounded provider transcript. Engineer and Reviewer use separate resumable sessions so recent context and provider prefix caches remain useful, while a structured `checkpoint.json` in the project state directory carries curated cross-round state (`engineer/checkpoint.py`, `apps/_runtime.py`). Its `CheckpointState` fields cover the goal, verified work, dead ends, maturing directions, one open blocker, the next step, an active-line code pointer, and bounded environment facts. Python enforces item and character caps. The Engineer ends a turn with a concise handoff proposal; when independent review runs, the Reviewer checks that proposal against the artifacts and authors the canonical checkpoint. One bounded review deferral is permitted by default when the Engineer returns an explicit next execution step; the accumulated work is then reviewed.

The resume window is explicitly bounded. By default, the runtime rolls each role's thread after three rounds on that thread, or when the preceding call reports at least 1.5 million input tokens; the next call opens a new session and the Engineer is reseeded from the checkpoint (`engineer/runner.py`, `core/knobs.py`). If provider-side compaction occurs before a roll, the next round resends the full static preamble. This combines short-window context reuse with a structural limit on transcript growth. The checkpoint remains a *current-state* object rather than an append-only log; stale detail is deleted rather than accumulated, while ground truth remains in on-disk artifacts.

6.3 Session model and recency memory

A run keys its per-project state by a session id: `-new` starts a fresh session, `-resume [<id>]` offers a picker, and `-continue` attaches to the most recent (`core/session.py`). Combined with the identity-checked daemon resume (Section 5), this controls whether a restart continues an existing campaign or begins a new one. Before each mission the scheduler also renders a short memory preamble from the most recent project journal entries, marked *non-authoritative advisory* (`life/memory.py`). The preamble is pure recency — it does not score "relevance" by keyword overlap; which past work matters is left to the agent to judge after reading. The raw objective is never mixed into it, so skill-matching stays clean.

6.4 Skills: reusable capability with a lifecycle

Skills are Markdown files with lightweight frontmatter, managed by a skill subsystem of roughly 33 modules (`argus_skill/skills/`). On a matcher miss a Scientist distills a new, immediately usable project-layer skill and saves it versioned; there is no separate quality gate or provisional/confirmed promotion state. A skill's authority comes instead from evidence: real usage records, the Reviewer's feedback, version history, and a reversible archive/compaction path feed later update, split, merge, and retire decisions (`skills/store.py`, `skills/scientist.py`, `skills/lifecycle.py`). The Reviewer is the *sole* author of skill operations for a mission — there is no Manager gate on a skill update. The core loop contract (distill on miss, then converge; short-circuit on `blocked`; stay bounded by `max_rounds`) is documented by `tests/test_loop_smoke.py`.

6.5 Project wiki: immutable, evidence-cited knowledge

The second knowledge system is a per-project wiki (`argus_skill/wiki/`, roughly 14 modules). After each real Reviewer verdict a *source* RoundCard is written mechanically from the verdict, planner report, and research result; these sources are immutable and citable. The Reviewer may later synthesize technique/conflict/pattern pages from them, but every page must cite evidence spans that are *mechanically verified verbatim* against a wiki source (`skills/provenance.py`, `verify_evidence`). A fabricated citation is rejected regardless of the Reviewer's judgment — the same fail-closed provenance rule that governs completion certification (Section 10). The Planner reads only the few most recent sources, so the wiki informs planning without becoming an unbounded prompt.

7 Agent Roles and Decision Interfaces

Design objective

Every scientific and scheduling decision crosses a narrow, typed interface — a Python dataclass with named fields — so that authority is explicit, verdicts are machine-checkable, and no role can quietly reach past its contract. Four persistent roles carry all judgment; the infrastructure only routes their typed outputs.

Argus runs four persistent roles. Three form a numbered execution spine — Planner (L4), Engineer (L1), Reviewer (L2) — while the Manager sits outside the numbering because it spans the whole pipeline rather than occupying a station on it. (A fifth role, the Curator, runs only in team/subagent mode and is out of scope here.) This section documents each role's *decision interface*: the typed contract it emits and the authority it holds.

7.1 Manager — front door and stage authority

The Manager converts an operator's free text into a committed campaign in a single grounded model call that emits three structured axes at once — `CONFIG` (e.g. a backend/model change), `CONTROL` (whether to dispatch at all), and `ROUTE` (chat vs. task and which vertical). A `CONTROL: NO_DISPATCH` decision is authoritative: the bridge forces a read-only self-reply and, if that fails, fails closed rather than enqueuing work. The result of a task division is a `Division` (task, chosen

vertical, stage template); stage progression is a `StageTransition`. The Manager is the sole writer of `current_stage`; the transition's `action` and `source` fields (Table 5) make both the decision and its justification auditable, including the several fail-safe holds that keep the pipeline coherent when a downstream role is unavailable.

Field	Type / values	Meaning
Division `manager/_core.py`		
`task`	str	the committed objective text
`vertical`	str	the one selected task shape (never re-classified)
`stages`	list	the vertical's ordered stage template
StageTransition `manager/_core.py`		
`action`	advance | hold | rollback | complete	the stage move applied to `current_stage`
`source`	manager_llm | *_hold	why: a model decision, or a fail-safe hold (no review / no runner / failsafe / illegal target)

Table 5. Manager decision interface. A `hold` writes nothing; only the Manager mutates `current_stage`.

7.2 Planner (L4) — forward scheduling

When the backlog empties in continuous mode, the Planner proposes the next batch of work as a set of `TaskSpecs` and returns a `PlannerVerdict` (Table 6). Tasks form an optional DAG: each `TaskSpec` has a local `key` and a `deps` list, which the scheduler maps to real backlog ids when it enqueues the batch (flat tasks leave both empty). Each task carries a structured `scope` — `bounded` or `final_submission` — that is threaded to the Reviewer as a backlog tag, so completion authority never has to re-parse prose to learn a task's scope. The verdict also has a first-class, intentional `waiting` outcome (with a durable `WaitingContract`: a blocker fingerprint, a recheck condition, and a token) for the case where the project is correctly blocked on a live external job and there is no genuinely new high-impact work to queue — an idle state that is neither an error nor make-work. The Planner owns the per-stage checklist and emits `checklist_ops` to edit it. The canonical verdict-status contract and its (success, recoverable) policy live in `core/planner_verdict.py` (Appendix A).

Field	Type	Meaning
TaskSpec `planner/planner.py`		
`title, objective`	str	human title + full actionable description
`impact_score`	int 0–5	parser accepts only high-value work
`scope`	bounded | final_submission	threaded to the Reviewer as a tag
`key, deps`	str, list	DAG node id and its dependencies
PlannerVerdict `planner/planner.py`		
`project_done`	bool	project-level completion proposal
`new_tasks`	list[TaskSpec]	the next batch
`waiting`	bool	intentional idle on a live external blocker
`waiting_reason`	str	paired with a `WaitingContract`
`restart_daemon`	bool	request a controlled daemon restart
`checklist_ops`	list	per-stage checklist edits (Planner owns it)

Table 6. Planner decision interface (abridged). Token-usage fields are omitted; scope is structured, never inferred from the objective text.

7.3 Engineer (L1) — one round of real work

The Engineer executes a single round against real tools and hardware. Its interface to a provider is the per-call `RunnerOptions` contract, and it receives back a `RunnerResult` (Table 7). Two properties of this contract matter for reliability and integrity. First, provider-side spend fences (`max_budget_usd`, `max_ai_credits`) are populated from an atomic budget *reservation*, not from role configuration, so a role cannot raise its own ceiling (Section 8). Second, the result carries usage-*presence* booleans so that a real zero token count is distinguishable from a provider that simply omitted usage — the evidence plane never invents a number a provider did not report. Watchdog and streaming hooks (`watchdog_soft/hard_idle_seconds`, `on_agent_message`) are honored by the subprocess backends and ignored by the deterministic test backend. The Engineer's own round-loop configuration (round bounds, stall thresholds, effective-progress timeout) is documented with the reliability guards in Section 9.

Field	Meaning
RunnerOptions (per-call backend knobs) `core/models.py`	
`model, reasoning_effort`	provider model and effort level
`output_schema_path`	structured-output schema for the call
`live_search`	provider web search; research stage only
`sandbox_mode`	containment policy for the spawned role
`max_budget_usd, max_ai_credits`	spend fences from a reservation, not config
`watchdog_*_idle_seconds, on_agent_message`	idle watchdog + streaming (subprocess backends)
RunnerResult (per-call outcome) `core/models.py`	
`call_id, call_id_log_correlated`	audit correlation to the event tape
`*_tokens_present`	presence flags: a real zero vs. an omitted usage
`premium_requests, total_nano_aiu, cost_usd`	usage and cost accounting
`pricing_status, tool_activity_observed`	pricing state; whether the round used tools

Table 7. Engineer per-call backend contract (abridged). Spend fences come from a reservation; presence flags keep usage accounting honest.

7.4 Reviewer (L2) — the completion authority

The Reviewer is the sole authority on completion, and there is no separate hardcoded completion gate anywhere in the system. Each round it inspects the Engineer's output against a structured stage checklist and returns a `ReviewDecision` (Table 8) whose `status` is `done`, `continue`, or `blocked`. Because the Reviewer runs in the same work-tree as the Engineer and is given the mission's execution log with grep recipes, it can audit *how* a result was produced — catching a hardcoded answer, a skipped step, or a faked metric — rather than trusting a summary (Section 10). The verdict is rich: `progress_class` is the Reviewer's structured judgment of forward motion (the infrastructure counts it, never infers it); `operator_question` is the single human-facing block question; `scope` plus `checklist` form the final-submission gate; `skill_ops` and `wiki_ops` are authored here with no Manager gate; and `backend_unavailable` flags an infrastructure failure so it is never mistaken for a `continue`. A `final_submission_certified` property is fail-closed — it is true only when the structured gate is genuinely satisfied.

Field	Meaning
`status`	`done` | `continue` | `blocked` (the completion verdict)
`progress_class`	`decision` | `evidence` | `setup_only` | `artifact_sync_only` | `none`
`next_action`	concrete next step handed to the subsequent Engineer round
`operator_question`	the one human-facing block question, if any
`scope, checklist`	final-submission gate (fail-closed)
`achievement, failure_cause`	certified achievement payload / diagnosed cause
`skill_ops, wiki_ops`	knowledge-system edits (Reviewer is sole author)
`backend_unavailable`	infrastructure-failure flag, not a judgment
`step_back`	anti-plan-lock-in signal

Table 8. Reviewer verdict — the completion contract (`core/models.py`, `reviewer/reviewer_schema.json`). All fields are required by the schema; the achievement payload is emitted only on a `done` verdict.

Why "done" has exactly one owner

Premature acceptance (Section 3) is answered structurally: completion is a single typed verdict from a role that sees the artifacts and the execution log, not a self-report from the role that did the work, and not a keyword the infrastructure fished out of a summary. If the Reviewer is wrong, that is a model-quality limitation (Section 15) — but the authority is never ambiguous, and it is never silently overridden.

7.5 What was deliberately removed

The commitment to keeping judgment with the agent is visible as deletions, not slogans. A prior reviewer path scanned the Engineer's summary with regexes and forcibly rewrote a `done` verdict into `continue`; it and its pattern constants were removed, and the underlying constraint now lives in the Reviewer's own prompt. Chat-vs-task routing is a low-effort model call, not a length or regex heuristic. And task type, lifetime, and scope are decided by explicit structured signals — a vertical's completion gate, a configuration flag, a backlog tag — not by grepping the objective text.

8 Experiment Execution and Resource Infrastructure

Design objective

Route every provider call through one gateway, price it, and charge it against an atomic budget reservation before the work runs — so that spend is bounded, attributable, and impossible for a role to escalate on its own. Long experiments run as supervised background jobs; scarce hardware is coordinated through an explicit lease.

8.1 One gateway for every call

All backend execution flows through a single application-level gateway, `RunExecGateway.execute` (`core/run_gateway.py`), which assigns a `call_id`, propagates the resume-thread identity, stamps timing, and returns a normalized `RunnerResult`. Because there is exactly one interception point, tracing, metrics, cost accounting, and policy apply uniformly regardless of role or provider (Codex, Claude Code, Copilot CLI); every provider interaction carries an id that ties it to the event tape, which is what makes the per-call audit of Section 10 possible.

8.2 Pricing and the usage ledger

Token usage is priced by a small pricing module (`core/pricing.py`) that maps a model to input/output rates (default model `gpt-5.5`) and quotes token usage into dollars, including the conversion of Copilot premium-request counts into USD. Priced usage is accumulated in a usage ledger (`core/usage.py`) as token counts, premium requests, a nano-AIU credit total, and cost. The ledger is *presence-aware*: it distinguishes a genuine zero from a provider that omitted usage (Section 7), so no cost is ever fabricated from missing data.

8.3 Budget reservations and spend fences

Spend control is reservation-based and event-backed. Before a call runs, the runtime creates a budget reservation; the outcome is recorded as one of `budget.reservation.created`, `budget.reservation.denied`, `budget.reservation.settled`, or `budget.reservation.released`, with `budget.unpriced.blocked` and `budget.fence_breach.blocked` for the cases where a call cannot be priced or would breach a fence (`core/event_catalog.py`). Crucially, the provider-side spend fences on a call (`max_budget_usd`, `max_ai_credits`) are injected from the reservation, not from role configuration — a role cannot raise its own ceiling. Table 9 lists the principal budget defaults, all verified against the live configuration.

Control	Default	Effect
Per-mission preflight cap (`per_mission_cap_usd`)	$30	max spend admitted for one mission
Daily cap (`daily_cap_usd`)	$180	max spend per local day
Global daily cap (`global_daily_cap_usd`)	0 (off)	cross-project daily cap; disabled unless set
Autonomous missions per interactive run (`max_missions`)	6	bound on one supervisor run (the daemon runs continuously)
Planner task-iteration budget (`..._BUDGET_USD`)	$30	per-task iteration budget surfaced to the web layer
Planner task-iteration cycles (`..._MAX_CYCLES`)	6	max planner iterations per task
Max active daemons (`ARGUS_SKILL_MAX_ACTIVE_DAEMONS`)	2	concurrency bound across projects

Table 9. Principal budget/quota defaults (`life/supervisor/_config.py`, `daemon/config.py`, `apps/cli/_core.py`). All are operator-overridable; the values shown are the live code defaults.

8.4 Provider quota and fencing

Beyond dollars, a provider may impose its own quota. A per-day Codex quota state is tracked in a lock-guarded state file (`core/provider_quota.py`); a call first acquires a `ProviderPermit`, and a denial carries a `stop_kind` such as `budget_exhausted` so the mission ends in a paused status rather than an error (Section 5). Related fencing and a Copilot-specific guard (`core/provider_fencing.py`, `core/copilot_guard.py`) keep a single provider outage from cascading into a spin.

8.5 Background jobs and teams

Not all work is short. When the Engineer starts a long experiment — for example a GPU training run — it launches a supervised background job that has its own monitor: the monitor checks health on an interval, early-stops on failure, and reports a terminal result to the operator inbox. The Engineer reads a registry of in-flight jobs (`engineer/background_subagents.py`) and classifies each as self-watched or needs-attention; the cadence-wait mechanism that lets it yield to a job's own monitor instead of busy-polling is described in Section 9. A separate team subsystem (`argus_skill/team/`, roughly 12 modules) supports parallel subagents with a Curator maintaining the shared skill pool; it is an optional mode, not part of the single-mission path.

8.6 Coordinating scarce hardware: the GPU lease

On managed GPU hosts a scheduler reclaims cards that sit idle, so an operator runs a low-duty "keep-alive" loader to hold them. Argus ships an explicit *lease* tool for this (`argus_skill/tools/gpu_lease.py`, with a standalone loader `argus_skill/tools/gpu_load.py`): the recommended path, `gpu_lease run -- <cmd>`, frees the cards, records a lease for the job's lifetime, runs the command, and on exit re-parks the keep-alive only if no other lease is active. A `-detach` mode lets a supervisor own the lease independently of the mission, and a watchdog re-parks the cards only once they are unheld, lease-free, and idle. The keep-alive's argv and cwd are recorded so the tool restarts the exact program the operator started, and process termination only ever targets the configured match token.

Scope of the GPU lease

The GPU lease is a real, robust *operator/agent-invoked* capability (`argus_skill/tools/gpu_lease.py`, `tools/setup.py`), not an automatic runtime service: nothing in the mission scheduler or engineer loop calls it implicitly. It is presented here as resource-governance tooling the agent may choose to use, which is what the live integration supports — no more.

9 Reliability, Recovery, and Long-Running Operations

Design objective

Treat execution drift — not weak reasoning — as the primary risk of a multi-hour run. Every degradation mode maps to a named guard with an explicit trigger and action; each guard is task-agnostic, fails soft, and is precise about whether it may interrupt a live provider call.

The reliability model is a set of guards that keep a mission bounded and honest. Figure 6 sketches how they wrap one Engineer/Reviewer loop. Table 10 is the guard matrix: for each guard, its trigger, its live default, its action, and — critically — whether it can interrupt a call that is still running. The distinction matters because a guard that severs useful in-flight work is as harmful as no guard at all.

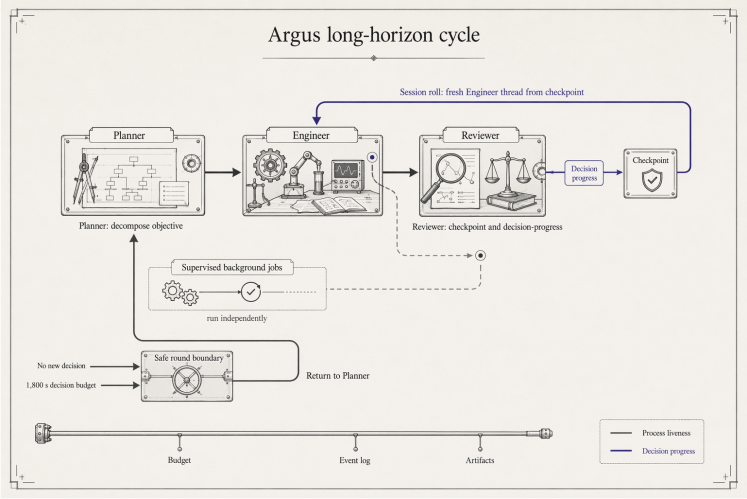


Figure 6. System schematic (not a measured performance plot) of the long-running reliability cycle: Planner, Engineer, and Reviewer run one loop; the Reviewer maintains a bounded curated checkpoint and a decision-progress class that guides the next Engineer round; supervised background jobs run alongside; and a safe round boundary returns a stalled or budget-exhausted mission to the Planner, over a persistent budget, event-log, and artifact rail.

9.1 The fault model

Argus recognizes six recurring fault classes and routes each to a first-class outcome rather than an unbounded spin. A *provider error or outage* triggers a bounded backoff and, if it persists, ends the mission as an error (a quota denial ends it as a paused status, Section 8). *Context pressure* is bounded by short-window session reuse plus proactive round- and token-triggered rollover (Section 6), with an in-round compaction guard for the rare repeatedly-compacting round. *Stale or self-watched jobs* are handled by the background-subagent cadence wait; *no-decision loops* by counting the Reviewer's `progress_class`; a *reviewer or backend failure* is treated as infrastructure, never a `continue`; and *process interruption* drains to the mission boundary with an identity-checked resume (Section 5).

9.2 The guard matrix

Guard	Trigger / action	Default	Interrupts live call?
Backend-failure recovery	failure streak → backoff, then end as error	2 / 15 s	no (round boundary)
No-progress bail	consecutive rounds with no engineer message	2	no
Decision stall	consecutive nondecision `progress_class` rounds → end	2	no
Decision-progress budget	seconds since last decision/evidence → end	1,800 s	**no**
Soft round escalation	round index → ask Reviewer to return `blocked`	12	no
Hard round escalation	round index → force-end as `blocked`	24	no
Effective-progress watchdog	no file change / non-token event → kill subprocess	3,600 s	**yes**
Runner hard-idle	no runner activity → interrupt round	3,600 s	**yes**
In-round compaction limit	auto-compactions within one round → end round	3	**yes**

Table 10. Reliability guards (`engineer/runner.py`). Round-boundary guards never sever a live call; the three in-round guards fire only against a subprocess that is emitting token noise or looping without effective progress. All are env-overridable and disable at 0.

9.3 Decision progress, not activity

To separate real progress from busy churn the Reviewer labels each round with a `progress_class`: `decision` or `evidence` for genuine forward motion, or `setup_only`, `artifact_sync_only`, or `none` for nondecision work. The runtime *counts* these structured labels; it never infers progress from tool activity, filenames, or token emission. Two consecutive nondecision rounds trip the stall guard, and the 1,800-second decision-progress budget — measured from the last decision or evidence increment and checked only between rounds — ends a mission that is spinning without ever cutting useful in-flight work. A separate soft/hard round escalation (12/24) catches the rarer case of a mission that makes evidence progress every round but never passes its gate, asking the Reviewer to return `blocked` on an external dependency and, failing that, force-ending so the Planner re-plans.

9.4 In-round watchdogs

Three guards operate *inside* a round because they target a subprocess that has stopped doing useful work. The effective-progress watchdog kills a provider subprocess that keeps emitting heartbeat or token noise for an hour with no file change and no non-token session event; the runner hard-idle watchdog interrupts a round with no runner activity at all; and the in-round compaction guard ends a round whose session auto-compacts three times (the in-round amnesia-loop signature). Each

default is intentionally long — research turns legitimately spend many minutes reading, planning, or waiting on model-side recovery — and each disables at 0.

9.5 Backend-unavailable is infrastructure, not judgment

A specific, high-cost failure motivated a dedicated path. If the reviewer backend degrades silently, the sole completion gate can run *blind*: a stale import-time schema path once caused every reviewer round to exit non-zero, running the completion gate blind for roughly an hour and a half before it was caught. The fix routes a backend-unavailable reviewer round to the structured `backend_unavailable` flag with transport fields, treated as an infrastructure failure and never as a `continue`; the reviewer also preflights its schema file and raises if it is unreadable rather than proceeding (`reviewer/_core.py`, `core/models.py`). This is the clearest illustration of the report's reliability posture: the failure was not that a model reasoned poorly, but that a degraded component was allowed to look healthy.

9.6 The single anti-stall mechanism

Layered above the per-round guards is exactly one anti-stuck mechanism, regime-jump (`regime_jump/`). It *detects* saturation with a simple counter, *asks* the Planner to judge whether to change regime, and *enforces* a never-cleared forbidden ledger of exhausted directions — and it fails soft to a no-op if anything in the mechanism itself goes wrong. It is a deliberate choice to have one such mechanism rather than many overlapping heuristics, each of which would be another place for the infrastructure to mis-judge the science. None of the guards in this section decides whether the research is good: each enforces a task-agnostic invariant and fails soft, preserving the agent's memory and judgment when anything goes wrong — reliability is added *around* the agent's judgment, never in place of it.

10 Evaluation and Evidence Architecture

Design objective

Make every reported number reconstructable and every completion auditable. The evidence architecture records the run on an append-only tape, audits how each result was produced, redacts credentials before persistence, hashes published artifacts for provenance, and labels every quantity by the strength of its evidence. A verdict may be wrong, but it can never be unfalsifiable.

10.1 The immutable event tape

The canonical record of a project is an append-only event tape, `events.jsonl`, written through typed events from a catalog of 112 event types across 11 categories, of which 75 carry a validated payload schema (`core/event_catalog.py`; Appendix C). Within it exactly one event, `research.achievement.certified`, is the authoritative record of a verified achievement, emitted only on a `done` verdict carrying the structured achievement payload (Section 7). A campaign's decisions, spend, and outcomes can be replayed from the tape after the fact, which is what makes an audit possible at all.

10.2 Execution-log audit for the Reviewer

Trusting a self-report would defeat the completion contract, so the Reviewer can check the Engineer's work directly. The two roles share a work-tree, and the Reviewer's prompt includes an audit section pointing at the mission's own `events.jsonl` with grep recipes (`engineer/runner.py`), so the Reviewer can inspect *how* a result was produced — detecting a hardcoded answer, a skipped step, a cheat method, or a faked metric — rather than accepting the summary. Each provider call is correlated to the tape by `call_id`, and usage-presence booleans keep a real zero distinct from a missing measurement (Section 7). This verifies the number is honest, not that the science is good.

10.3 Anti-cheat by construction

For benchmarks whose metric could be gamed by memorizing a fixed evaluation input distribution, the evaluation input is randomized per run so a solution cannot hardcode the statistics of a known distribution — a fast-because-faked kernel is designed to be impossible to pass off as genuinely fast. Combined with real public benchmark data and the per-round audit trail, this closes the metric-exploitation failure mode by construction rather than by after-the-fact inspection.

10.4 Credential redaction before persistence

A task-agnostic secret guard (`core/secret_guard.py`) scrubs credentials from newly written artifacts before they reach the event log or the Reviewer's prompt, and redacts persisted event and usage copies, matching authorization and API-key patterns, provider tokens, and generic key/value secrets. Redaction never judges scientific quality; when it rewrites an artifact it emits `round.secret_redacted` so the Reviewer can rebuild the affected provenance hashes, and a scan error or an oversized artifact explicitly *blocks* unconditional certification rather than silently passing — the same fail-closed stance as the completion gate.

10.5 Provenance and artifact corroboration

Published evidence carries provenance. For website-sourced results, the retrieval records an HTTP status and a raw-HTML SHA-256 for each source page; for corroborated results, the bundle identifies an artifact in a named external project by logical project name, artifact id, and SHA-256 — with no local absolute paths, usernames, or session ids. The artifact bytes are not committed in this repository (`technical_report/evidence/README.md`). The report figures carry their own provenance sidecars and reproducible hashes (Appendix D). Result *signing* is deliberately not claimed: no cryptographic result-signing component exists in the current tree, and the report does not imply one.

10.6 Evidence tiers

Every quantitative statement is labeled by the strength of its evidence, and the label travels with the number wherever it appears. The evidence bundle defines two concrete tiers; the report additionally distinguishes external baselines and ongoing programs as reporting categories (Table 11).

Corroboration / category	What it means	What it may claim
`local_artifact`	A website result with a committed digest record for an artifact in a named external project (logical project + artifact id + SHA-256).	An Argus result for that run, on stated hardware.
`website_snapshot`	A result card quoted verbatim from the live public record, with no artifact-digest corroboration yet.	A scoped public claim under its stated protocol.
External baseline	A published third-party or human target used for comparison.	A reference target; explicitly *not* an Argus result.
Ongoing / no claim	A program running without certified, reproducible completion.	Status "ongoing"; no numeric Argus claim.

Table 11. Evidence categories. All six claims originate as `website_snapshot`; the bundle's separate `corroboration` field marks two as `local_artifact` because digest metadata is available (Section 11).

10.7 Completion is a single certified verdict

The measurement-integrity architecture converges on one rule: a program is complete only when the Reviewer certifies it against the full-pipeline stage checklist, with the `final_submission_certified` property fail-closed. There is no hardcoded completion gate and no independent validator behind

the Reviewer; an earlier family of roughly twenty machine "validate-*" quality gates was removed once completion authority moved to the Reviewer's checklist, so that completion has exactly one owner and one auditable record. This concentrates authority in a single decision point — examined honestly in Section 15 — but it also means completion is never the product of two components silently disagreeing.

11 Public Results Across Six Arenas

Design objective

Report each public result as a scoped claim under the exact protocol it was measured under, using human or paper-reported references first where available, with its evidence status attached. Units differ across arenas and are never cross-normalized; a website snapshot is never presented as a reproduced artifact.

Argus publishes a live public record [1]. This section reproduces its six current arena results. Figure 7 presents them as small multiples — one panel per arena, each with its own unit — because the arenas measure different quantities (kernel rank, bits-per-byte, wall-clock seconds, solve rate, a gap score) and any shared axis would be misleading. Table 12 gives the exact protocol, result, published comparison, and evidence status for each row.

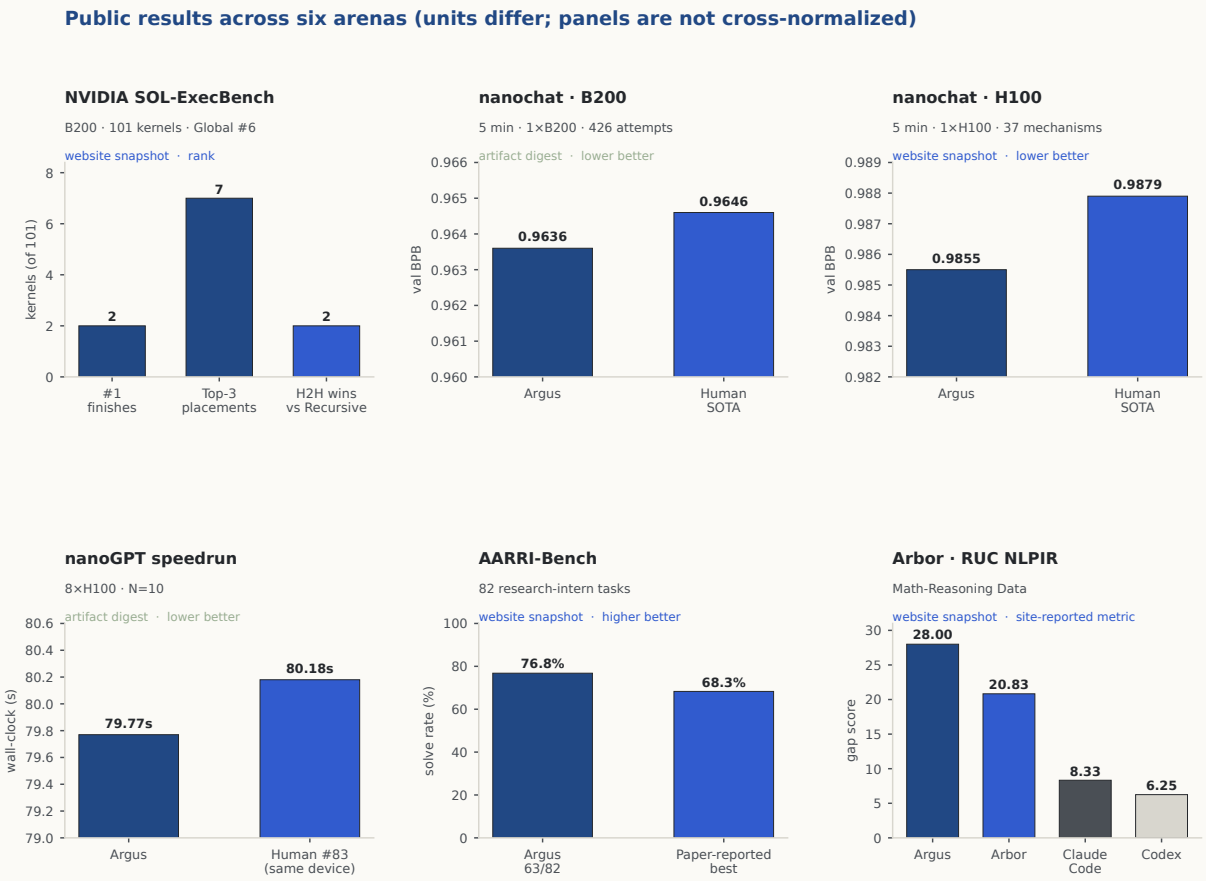


Figure 7. Public results across six arenas, drawn deterministically from `technical_report/evidence/website_results.json`. Panels are not cross-normalized; each carries its own unit and evidence status. Direction is shown where defined; Arbor retains the site's reported metric without inferring its direction. Argus bars are indigo; human/system baselines are lighter.

Arena	Protocol	Argus result	Published comparison	Status
NVIDIA SOL-ExecBench	B200; 101 kernels	Global #6; 2×#1; 7 top-3	Global rank vs. all entrants (headline)	snapshot
nanochat (B200)	5 min; 1×B200; 426 attempts	0.9636 BPB	Human SOTA 0.9646	**digest**
nanochat (H100)	5 min; 1×H100; 37 mechanisms	0.9855 BPB	Human SOTA 0.9879	snapshot
nanoGPT speedrun	8×H100; N=10	79.77 s	Same-device human #83: 80.18 s	**digest**
AARRI-Bench	82 research-intern tasks	63/82 (76.8%)	Paper-reported best 68.3%	snapshot
Arbor (RUC NLPIR)	Math-reasoning data	28.0 gap	Arbor 20.83; Claude Code 8.33; Codex 6.25	snapshot

Table 12. The six public results [1], verbatim from the evidence bundle. Human and paper-reported references are primary where available; the SOL-ExecBench headline is its global rank. SOL-ExecBench and Arbor also preserve the site's published system/agent comparisons. "digest" = committed artifact metadata; "snapshot" = public website snapshot.

11.1 Reading the arenas

Each number means exactly what its protocol row says and nothing more. On **SOL-ExecBench** [6] the headline is a *global rank* (#6) against all entrants on 101 B200 kernels, with two #1 finishes and seven top-3 placements; the two head-to-head wins over Recursive [7] are the site's note, not the framing. **nanochat** [5] is reported on both B200 (0.9636 BPB against a 0.9646 human SOTA) and H100 (0.9855 against 0.9879), each under a 5-minute training budget — lower bits-per-byte is better. The **nanoGPT speedrun** reproduces a fast GPT-training loop [4] on 8×H100 and reports 79.77 s over N=10 against the same-device human record of 80.18 s. **AARRI-Bench** reports 63 of 82 research-intern tasks (76.8%) against a paper-reported best of 68.3%. For **Arbor**, the website reports a "gap" value of 28.0 alongside Arbor 20.83, Claude Code 8.33, and Codex 6.25. The snapshot does not define the metric's direction, so this report quotes the published values without interpreting the ordering.

11.2 Artifact-digest corroboration versus website snapshot

Two of the six rows carry digest corroboration, and the difference is stated rather than smoothed over. For the **nanoGPT** 79.77 s row, the bundle references an N=10 verifier-certified artifact in the external `nanogpt-speedrun-h100` project — the verifier line records `valid=true, n=10`, a validation loss of 3.2776, a one-sided $p(\text{mean} < 3.28) = 0.0040$, a train time of 79.77 ± 0.06 s, and a `seal=ok`. For the **nanochat B200** 0.9636 row, the bundle records frozen-scorer, single-seed evidence of `MEAN_VAL_BPB` = 0.963634 with a frozen-scorer exit code of 0 and a genuine SM100 flash-attention kernel, in the `nanochat-mission-b200` project. This repository commits the logical artifact IDs, verifier excerpts, and SHA-256 digests, not the external artifact bytes. The other four rows — SOL-ExecBench, nanochat H100, AARRI-Bench, and Arbor — are `website_snapshot` with no artifact-digest corroboration yet, linked to the machine-readable bundle `technical_report/evidence/website_results.json`. No corroboration file was fabricated where artifact-digest metadata was not available.

What these results do and do not claim

Each row is a real, scoped claim under its stated protocol and evidence tier. The table does *not* claim universal state of the art, correctness outside these protocols, or that a snapshot is a reproduced artifact. Converting the four snapshot rows into artifact-digest records tied to a named commit is an explicit roadmap item (Section 16).

12 Research Portfolio: 41 Papers Across Six Programs

Design objective

Report the research output as a de-duplicated inventory, broken down by program and by manuscript/draft status, compared only to human-authored literature and baselines. Count what exists; claim no acceptances.

Beyond the benchmark arenas, Argus runs an optional idea-to-submission research pipeline (the `research` vertical), which drives literature grounding, experiment design, execution, results analysis, and manuscript writing through per-stage checklists and a Reviewer peer-review gate. Its public output is a collection of **41 papers** [2]: 35 manuscripts and 6 drafts, with duplicate versions removed, across six research programs. Figure 8 and Table 13 give the breakdown.

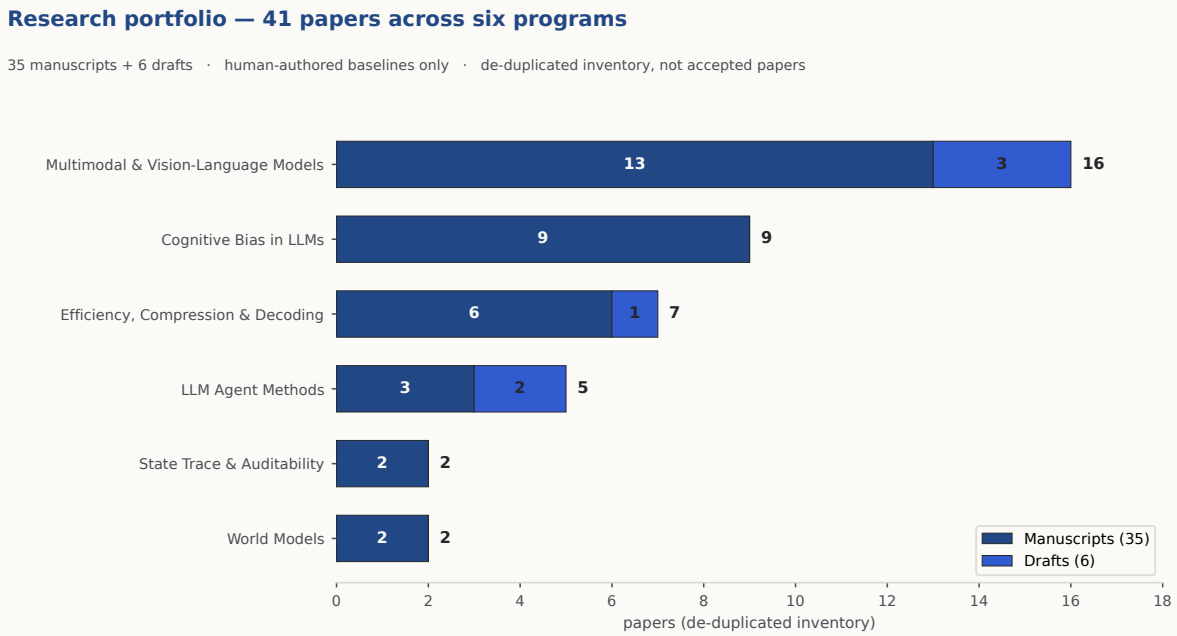


Figure 8. The 41-paper portfolio by program and status, drawn deterministically from `technical_report/evidence/paper_inventory.json`. Manuscripts are indigo, drafts lighter. The count is a de-duplicated inventory compared only to human-authored literature/baselines — not a count of accepted papers.

Program	Manuscripts	Drafts	Total
Multimodal & Vision-Language Models	13	3	16
Cognitive Bias in LLMs	9	0	9
Efficiency, Compression & Decoding	6	1	7
LLM Agent Methods	3	2	5
World Models	2	0	2
State Trace & Auditability	2	0	2
Total	**35**	**6**	**41**

Table 13. Research portfolio by program and status [2]. The machine-readable inventory is `technical_report/evidence/paper_inventory.json`.

12.1 The six programs

The portfolio concentrates in a few areas. **Multimodal & Vision-Language Models** is the largest program (16 papers), followed by **Cognitive Bias in LLMs** (9), which studies train-

free mitigation of biases such as anchoring, base-rate neglect, and status-quo defaults. **Efficiency, Compression & Decoding** (7) covers inference-time methods; **LLM Agent Methods** (5) covers agent techniques; and two smaller programs, **World Models** (2) and **State Trace & Auditability** (2), round out the set. Each paper is positioned against human-authored literature and baselines; the human comparison, where the site provides one, is preserved in the inventory.

12.2 What the count means

Three qualifications keep the number honest. First, 41 is a *de-duplicated inventory* of research runs that produced a manuscript or draft; it is not a count of accepted papers, and this report makes no acceptance claim. Second, the manuscript/draft split (35/6) is reported as-is: a draft is an in-progress artifact, not a finished manuscript. Third, the collection is compared *only* to human-authored literature and baselines — paper count and paper quality are never compared to any other agent or model. The research pipeline is a capability that produces auditable manuscripts under the same completion authority as every other mission (Section 7); it is not a claim that any of those manuscripts has cleared external peer review.

13 Operator Workbench and Observability

Design objective

Give the operator a live, read-then-command surface over the same event tape the system runs on: follow any mission round by round, inspect exactly what was produced, answer the one question a mission needs, and steer the daemon — all behind an authentication boundary on every state-changing action.

Argus is not only a daemon; it is a live workbench. An operator configures the machine's policy once — author identity, per-role backend and model access, GPU allocation — and then works through a cockpit backed by the same event stream and persisted state described in Sections 6 and 10. The cockpit has a terminal surface (an Ink TUI), a web surface, and an HTTP/WebSocket API (`webapi/server.py`, roughly 2,500 lines); there is no embedded Python REPL.

13.1 The four surfaces

Cockpit.
The front door routes free text through the Manager, so an operator can say "switch to the Copilot backend" in plain language and it takes effect without editing a config file.

Mission view.
The four roles are shown as peers, and a mission is followed round by round via `-status`, `-watch`, and `-follow`, backed by the structured event stream (`round.main.completed`, `round.review.completed`, and others).

Inspectable artifacts.
Because the Engineer and Reviewer share a work-tree and every round is persisted, the operator can open exactly what was produced and read the Reviewer's reasoning rather than a summary.

Intervention.
The operator can nudge or abort a mission, drain the daemon at a clean boundary, and rely on identity-checked resume so an upgrade does not silently re-plan (Section 5).

13.2 The API surface

Table 14 summarizes the API. Most project inspection endpoints are unauthenticated; commands, the live WebSocket, and global metrics and trash views pass through the authentication dependency and require an `Authorization` header when a token is configured (`webapi/server.py`). Routine run inspection therefore remains friction-free while mutation and sensitive operational views are gated.

Method	Representative endpoints	Auth
GET	`/api/projects`, `.../snapshot`, `.../events`, `.../status`, `.../journal`, `.../transcript`, `.../doctor`, `/api/meta`	open (read)
GET	`/api/metrics`, `/metrics`, `/api/trash`	token
POST	`.../tasks`, `.../message`, `.../nudge`, `.../plan`, `.../mission/abort`, `.../reset`, `.../skills`, `.../config/set`, `.../continuous`	token
POST	`.../daemon/start`, `.../daemon/stop`, `.../daemon/replace`, `.../daemon/upgrade`, `/api/daemons`	token
POST	`.../backlog/{item}/stop`, `.../backlog/{item}/dispose`, `.../identity`, `.../launch-cwd`, `.../note`	token
PATCH / DELETE	`/api/projects/{sid}` (rename / trash-restore lifecycle)	token
WS	`/api/projects/{sid}/stream` (live event stream)	token

Table 14. Cockpit API surface (representative; `webapi/server.py`). Most project inspection is open; commands, streams, metrics, and trash views are authentication-gated when a token is configured.

13.3 Event streaming and the operator inbox

A plain command POST discards the live agent-message blocks a mission emits, so a dedicated `/message/stream` endpoint surfaces them as they arrive, driven by the `on_agent_message` streaming callback (Section 7); the WebSocket stream carries the same typed events the tape records, so live viewing and later audit see the same data. The decision interface to a human is deliberately narrow: a mission surfaces at most one blocking question — the Reviewer's `operator_question` — queued to and drained from an operator inbox (`life.inbox.queued/drained`), answered through the command surface. This keeps unattended operation genuinely unattended: the system asks for input only when a mission is truly blocked on a decision only a human can make.

14 Deployment, Security, and Cost Control

Design objective

Deploy as a standard, inspectable service that survives crashes and reboots and drains cleanly on stop; contain each builder role to its own project working directory; keep credentials out of both artifacts and manuscripts; and bound spend with reservations rather than trust. Prefer boring, auditable mechanisms over bespoke supervision.

14.1 Deployment topology

The intended deployment is a per-project daemon supervised by `systemd -user`. The repository ships a unit template (`deploy/argus-skill.service`) whose semantics are summarized in Table 15: it starts the daemon in the foreground so the service manager owns the process lifecycle, drains to the mission boundary on stop, restarts on failure with a bounded restart budget, and is cwd-bound so the working directory maps to a project fingerprint. Running `loginctl enable-linger` carries the daemon across logout and reboot. Crash and reboot restart is thus owned by a standard, inspectable unit file rather than by custom logic; the identity-checked resume (Section 5) ensures a restart continues the right campaign.

14.2 Containment: sandbox and safe mode

Each builder role (Engineer, Reviewer, Planner, subagent) can be run under a provider sandbox. The sandbox policy (`core/sandbox.py`) resolves a workspace-write sandbox confined to the project working directory, or an *honest* bypass of `None` when no real sandbox is available — and it *refuses* a fake sandbox that would present as confined while remaining writable. The containment invariant is explicit: a sandboxed builder may write only its project working directory, never the global state root (`~/.argus-skill`) or the provider's own configuration, because writes there would let a role

Unit directive	Value / effect
`ExecStart`	`argus-skill -daemon-fg` (foreground; systemd owns lifecycle)
`ExecStop`	`argus-skill -daemon-stop -drain` (quiesce to mission boundary)
`TimeoutStopSec`	`1800` — let a draining mission finish before escalating
`KillSignal`	`SIGTERM` (no mid-mission `SIGKILL`)
`Restart` / `RestartSec`	`on-failure` / `10 s`
`StartLimitIntervalSec` / `Burst`	`300` / `5` — bound restart storms on a bad config
`WorkingDirectory`	the project worktree (cwd → fingerprint)

Table 15. The `systemd -user` unit template (`deploy/argus-skill.service`). The unit owns crash/reboot restart; stop drains rather than kills.

edit the very controls that supervise it. A separate `ARGUS_SKILL_SAFE_MODE` gate is honored by the Manager and the supervisor. The sandbox is opt-in per box because network, cache, and cluster access must first be verified under it.

14.3 Secret handling

Security of credentials is handled in two places. Before persistence, the secret guard (Section 10) scrubs credentials from artifacts, events, and usage copies, and blocks unconditional certification on a scan error or an oversized artifact. Before a run, a vault preflight (`core/vault_preflight.py`) checks secret availability. For the research pipeline specifically, a dedicated review pass looks for infrastructure leakage — local paths, device or cache details, or provider route/config — so that runtime detail never reaches a manuscript. None of these is a proof of zero leakage; each is a task-agnostic scanner, and the operator remains responsible for how secrets enter the environment.

14.4 Cost control

Spend is bounded by the reservation-based budget system of Section 8: a per-mission preflight cap ($30 by default), a daily cap ($180), an optional global daily cap (off by default), and provider-side spend fences injected from an atomic reservation rather than from role configuration. Because the fences come from the reservation, a role cannot raise its own ceiling, and because every reservation is event-backed, spend is fully attributable after the fact. These are defaults, not guarantees: a poorly bounded campaign can still be expensive, and 7×24 operation is not free (Section 15).

15 Limitations and Open Engineering Problems

Argus gives an agent real judgment, real tool access, and real persistence. It does not guarantee good science, and a report that documented reliability mechanisms without documenting their boundaries would itself be misleading. The limitations below are material, and several are open engineering problems the roadmap (Section 16) targets directly.

15.1 Model and objective ceilings

Provider dependence.
Argus drives an external agent CLI (Codex, Claude Code, or Copilot CLI) and inherits that provider's capability, availability, latency, and pricing. An outage, a regression, or a rate limit is felt directly, and the quality ceiling of any run is the underlying model's.

Objective quality is the operator's.
The system optimizes the objective it is given. A vague or trivially gameable objective yields vague or gamed work; Argus enforces integrity of *measurement*, not wisdom of *goal*.

No guarantee of novelty, correctness, or SOTA.
Nothing in the runtime guarantees that an idea is novel, that a proof or result is correct, or that a number is state of the art. Those are properties of the agent's judgment and the problem's

difficulty, and the system deliberately does not fake them.

15.2 Sole-reviewer completion authority

The Reviewer is the single completion authority, and it is a model. A wrong Reviewer can accept an incomplete result or reject a finished one. Argus reduces this risk by giving the Reviewer the shared work-tree, the execution-log audit, and a structured checklist (Section 10), and by routing a degraded reviewer backend to an explicit infrastructure-failure status rather than a `continue`. But it cannot eliminate model error, and there is no independent oracle behind the Reviewer. The historical blind-gate incident (Section 9), in which a stale schema path ran the completion gate blind for roughly ninety minutes, is the concrete failure mode of concentrating authority in one component; the fix hardened the transport, not the underlying concentration of authority, which remains an open problem.

15.3 Evidence scope and integrity coverage

Four of six results are snapshots.
Only two of the six public results (Section 11) carry committed digest records referencing artifacts in named external projects; the artifact bytes are not in this repository. The other four are public website snapshots linked to the evidence bundle. They are real, scoped claims under their protocols, but they are not yet independently reproducible from a named commit, and the report does not present them as if they were.

Integrity is per-benchmark and ongoing.
The anti-cheat construction is task-agnostic, but a determined agent explores the exploit surface of each specific benchmark. A new benchmark can present a new exploit — a leaked test set, an unrandomized input, a side channel — that the current checks were not written for. Measurement integrity is an ongoing engineering surface, not a solved problem.

Paper inventory is not acceptances.
The 41-paper portfolio (Section 12) is a de-duplicated inventory compared to human-authored literature; it is not a count of accepted papers, and no acceptance is claimed.

15.4 Operational limitations

Cost.
Long-horizon autonomy consumes provider tokens continuously. Budgets and reservations bound spend, but 7×24 operation is not free.

Secret handling is a scanner, not a proof.
The credential guard scrubs artifacts before they are recorded, but it is a task-agnostic scanner: a scan gap or an oversized artifact blocks certification rather than guaranteeing zero leakage, and operators remain responsible for how secrets enter the environment.

Back-compatibility surface.
Legacy shims (an older global-directory variable and queue helpers) are kept for migration; they are technical debt to retire and add surface an audit must account for.

GPU lease is invoked, not automatic.
The GPU lease (Section 8) is robust but is an operator/agent-invoked tool, not an automatic runtime service; coordinating scarce hardware still depends on the agent choosing to use it.

15.5 Responsible operation

Given these limitations, Argus is meant to be operated and audited, not fired and forgotten. Responsible use means giving it well-posed objectives and honest benchmarks; setting budgets deliberately; reading the Reviewer's reasoning rather than a headline; treating every number according to its evidence tier (Table 11); and publishing only what the stated evidence supports. The system makes this practical — persisted artifacts, a round-by-round audit trail, a single inspectable completion authority — but the discipline of honest reporting ultimately rests with

the operator.

Maturity

Argus is under active development at Technical Report 0.3. The architecture and mechanisms described here are implemented and grounded in the current source tree; the public results (Section 11) are scoped claims under their stated protocols and evidence tiers, not universal-capability guarantees. Treat any performance number according to its tier — an artifact-digest record, a public website snapshot linked to the bundle, or a labeled external baseline — never as a general guarantee of capability.

16 Roadmap

The roadmap follows directly from the limitations in Section 15. Each item is an engineering objective, not a capability promise, and each is stated so that its completion is checkable.

16.1 From snapshot to reproducible evidence

The highest-priority item is to convert the four website-snapshot results of Table 12 — SOL-ExecBench, nanochat H100, AARRI-Bench, and Arbor — into artifact-digest records tied to a named commit, matching the nanoGPT and nanochat-B200 rows that already carry digest corroboration. The target is that every published number can make a specific, auditable claim on stated hardware, verifiable by a third party from a commit and an artifact digest.

16.2 Strengthening measurement integrity

Integrity coverage grows per benchmark: the plan is to extend the anti-cheat construction as each new arena reveals its exploit surface, and to grow the structured stage checklists the Reviewer verifies against — rather than re-introducing a separate family of machine validators that could silently disagree with the completion authority. The constraint is to keep exactly one owner of completion (Section 10) while widening what it is equipped to check.

16.3 Reducing the single-authority risk

The sole-reviewer concentration (Section 15) is the most important open problem. Near-term work reduces Reviewer error through richer execution-log auditing and evidence-grounded verdicts, and hardens detection of a degraded reviewer backend so a blind gate cannot recur. Whether to add independent cross-checks without recreating a hardcoded gate is an open design question rather than a settled plan.

16.4 Retiring debt and lowering cost

The back-compatibility surface (legacy shims and queue helpers) is scheduled for removal once migration paths are confirmed, and the GPU lease is a candidate for tighter but still explicit scheduler coordination. In parallel, tightening budget accounting and the cockpit — better cost attribution, clearer paused/blocked surfacing, lower-overhead observation — makes a 7×24 campaign cheaper to run and easier to steer.

The through-line

Argus today is a research infrastructure for reliable, auditable autonomy at version 0.3. The roadmap is the path from a trustworthy runtime to trustworthy *results* — reproducible evidence, broader integrity coverage, a less concentrated completion authority, and a cheaper campaign — pursued without letting the infrastructure substitute its judgment for the agent's, or any number outrun its evidence.

A Key Interfaces and Status Taxonomy

The role decision interfaces are summarized in Section 7 (Tables 5–8). This appendix records the status taxonomy they resolve into. Three enumerations govern outcomes: `ReviewStatus` (the verdict), `LoopStatus` (a mission's terminal or paused outcome), and `PlannerVerdictStatus` (the planner's event contract), the last carrying an explicit (success, recoverable) policy that determines how the scheduler reacts.

PlannerVerdictStatus	success	recoverable	Meaning
`planned`	yes	no	a new task batch was produced
`completed`	yes	no	the project is certified complete
`research_incomplete`	no	yes	more research is required
`paused_budget`	no	yes	budget exhausted; recheck later
`paused_no_breakthrough`	no	yes	no breakthrough this cycle
`exhausted_current_-methods`	no	yes	current methods exhausted
`provider_cooldown`	no	yes	provider cooldown in effect
`provider_fence`	no	yes	provider spend fence hit
`infra_blocked`	no	yes	infrastructure blocker
`error`	no	no	unrecoverable error

Table 16. `PlannerVerdictStatus` and its (success, recoverable) policy (`core/planner_verdict.py`). Legacy aliases (`done`, `paused_provider_cooldown`, `paused_provider_fence`) map onto these.

ReviewStatus (`core/models.py`): `done`, `continue`, `blocked`, plus the research-pause states `research_incomplete`, `paused_no_breakthrough`, and `exhausted_current_methods`.

LoopStatus (`core/models.py`): the terminal set `done`, `max_rounds`, `blocked`, `no_progress`, `error`, `budget_exhausted`; and the paused set `paused_budget`, `paused_provider_cooldown`, `paused_provider_fence`, `infra_blocked`, `research_incomplete`, `paused_no_breakthrough`, `exhausted_current_methods`; and the control-transfer status `replan_requested`, used only when dynamic planning requests a return to the Planner. Every mission yields one of these statuses — never an unbounded spin (Section 5).

B Persisted State Map

Table 4 (Section 6) lists the principal state files; the root layout is defined in `core/paths.py`, while runtime composition adds the per-project checkpoint path in `apps/_runtime.py`. The global root `~/.argus-skill/` (overridable via `ARGUS_SKILL_HOME`; a legacy `ARGUS_SKILL_LIFE_DIR` shim is retained for migration) holds cross-project identity, the shared skill library, and an operator knob store; each `projects/<fingerprint>/` subtree holds the append-only `events.jsonl`, `backlog.jsonl`, `continuous.json`, `inbox.jsonl`, a `skills/` directory, `daemon.pid`, `daemon.status.json`, and `checkpoint.json`. A runtime path resolver rejects unresolved shell placeholders, and the project-root helper rejects empty, `.`, and `/` fingerprints, so a mis-resolved path fails loudly rather than writing to the wrong place.

C Event Taxonomy

Cross-component state is exchanged through a canonical catalog of **112 typed event types** across **11 categories** (`core/event_catalog.py`); **75** carry a validated payload schema. Table 17 gives the per-category counts and representative events.

D Evidence Map

Every engineering claim in this report is grounded in the current `argus-skill` repository [3] rather than in transient run directories. Table 18 maps each subsystem to its primary live source.

Category	Count	Representative events
lifecycle	40	loop.start, round.main.completed, round.review.completed, budget.reservation.*, life.mission.started
skill	20	skill.match.*, scientist.start, skill.outcome
wiki	17	wiki source/page ingest, promotion, validation
planner	9	life.planner.start, life.planner.verdict
research	7	research.achievement.certified (sole authoritative achievement)
agent_io	5	agent.io.start, agent.io.stream, agent.io.complete
daemon	5	daemon start / stop / handoff / status
idea	3	idea lifecycle
provider	3	provider.request.started/completed/denied
usage	2	usage.recorded, cost
operator	1	operator inbox event

Table 17. Event taxonomy: 112 types across 11 categories (core/event_catalog.py). Budget, loop, and round events are grouped under lifecycle.

Subsystem (section)	Primary source path(s)
Three planes (§4)	docs/ARCHITECTURE.md; core/, manager/, engineer/, reviewer/
Mission lifecycle / daemon (§5)	life/supervisor/_core.py, daemon/life_worker.py, daemon/handoff.py
State & memory (§6)	core/paths.py, apps/_runtime.py, engineer/checkpoint.py, life/memory.py, skills/, wiki/
Role interfaces (§7)	core/models.py, planner/planner.py, manager/_core.py, reviewer/reviewer_schema.json
Execution / budgets (§8)	core/run_gateway.py, core/pricing.py, core/usage.py, core/provider_quota.py
Reliability guards (§9)	engineer/runner.py, regime_jump/
Evidence & integrity (§10)	core/event_catalog.py, core/secret_guard.py, skills/provenance.py
Cockpit / API (§13)	webapi/server.py
Deployment / security (§14)	deploy/argus-skill.service, core/sandbox.py, core/vault_preflight.py
Public results & portfolio (§11–12)	technical_report/evidence/website_results.json, paper_inventory.json

Table 18. Evidence map: subsystem to live source. This report cites repository paths on main, not any transient local filesystem location.

Figure provenance. The four deterministic figures (system_planes, mission_lifecycle, public_results, paper_portfolio) are generated deterministically by technical_report/figures/build_report_figures.py, with no image-model call. The two result figures read the committed evidence bundles; the architecture and lifecycle schematics encode source-grounded system structure. Reproducible SHA-256 digests are in figures/REPORT_FIGURES.json. The two editorial illustration plates were produced through the image-2 pipeline and carry prompt, inspect, review, and provenance sidecars in figures/IMAGE2_FIGURES.json.

E Paper-Inventory Summary

The 41-paper portfolio (Section 12) is enumerated in the bundle technical_report/evidence/paper_inventory.json, retrieved from the public research page [2] with an HTTP-200 raw-HTML SHA-256 for provenance. It totals 41 papers — 35 manuscripts and 6 drafts, duplicates removed — across Multimodal & Vision-Language Models (16), Cognitive Bias in LLMs (9), Efficiency, Compression & Decoding (7), LLM Agent Methods (5), World Models (2), and State Trace & Auditability (2), compared only to human-authored literature/baselines;

no external acceptance is claimed.

References

[1] Argus Team. Argus public results. Website snapshot, https://argusbot.cn/results.html, 2026.

[2] Argus Team. Argus research-paper collection. Website snapshot, https://argusbot.cn/research.html, 2026.

[3] Argus Team. argus-skill: A 7×24 system for long-horizon autonomous research. GitHub repository, 2026.

[4] Andrej Karpathy. nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. GitHub repository, 2022.

[5] Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy. GitHub repository, 2025.

[6] Anne Ouyang et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.

[7] Recursive. First steps toward automated AI research. GitHub repository, 2026.